

Ensemble Methods

TP – Risk certificates for ensemble methods – Solutions

M2 Computer Science – MALIA

Guillaume Metzler

Institut de Communication (ICOM)
Université de Lyon, Université Lumière Lyon 2
Laboratoire ERIC UR 3083, Lyon, France

`guillaume.metzler@univ-lyon2.fr`

Abstract. In this practical session you will compute PAC-Bayesian risk certificates for random forests, Gibbs posteriors and boosted ensembles on the datasets of the course, learn the weights of majority votes by minimizing these certificates, and use them for model selection. The session relies on notebooks 1 to 4 and on the module `pbtools.py`. Each part refers to the corresponding section of the lecture notes. At the end of the session you should have a short report comparing test risks and certificates, in the same format as in the first practical session. All the numbers of these solutions are produced by the script `tp_solutions.py` (also available as a notebook), with $\delta = 0.05$, 5 random stratified 70/30 splits and forests of 100 trees trained on bootstrap samples of size $m/2$; your numbers will differ slightly with other seeds.

Part 1 – Setting up

- Download the datasets (`datasets.zip`) and the loading script `prepdata.py` of the first practical session, and the module `pbtools.py`. The function `data_recovery(name)` returns a pair (X, y) with labels in $\{0, 1\}$: convert them to $\{-1, +1\}$ and standardize the features.
- Read `pbtools.py`: every function is short. It provides the binary `kl` and its inversions (`kl_bin`, `kl_inv_upper`, `kl_inv_lower`), the classical bounds (`bound_seeger`, `bound_mcalister`, `bound_catoni`, `bound_lambda`), a random forest keeping track of its out-of-bag samples (`BaggedTrees` with `oob_losses`, `oob_tandem`, `oob_disagreement`), the majority-vote certificates (`fo_bound`, `tnd_bound`, `cbound_pb`) and two optimizers (`optimize_fo`, `optimize_tnd`).
- Use **at least 10 datasets**, balanced and imbalanced, for instance `australian`, `balance`, `bupa`, `german`, `heart`, `iono`, `pima`, `sonar`, `wdbc`, `spambase` (balanced group) and `abalone8`, `pageblocks`, `satimage`, `segmentation`, `yeast3` (imbalanced group).

Solution

Loading helper used throughout the solutions:

```
def load(name):  
    X, y = data_recovery(name)  
    X = StandardScaler().fit_transform(np.asarray(X, dtype=float))  
    return X, np.where(np.asarray(y) > 0, 1, -1)
```

The sizes range from 208 (`sonar`) to 6435 (`satimage`) examples, and the proportion of positive examples from 2% to 48%. Keep these sizes in mind: a PAC-Bayesian certificate is computed with the number of examples on which the voters are evaluated, and with only a few hundred examples it cannot be tight.

Part 2 – Warm-up: inverting the kl and certifying one classifier (Section 2)

1. Implement the upper inversion $\bar{\text{kl}}^{-1}(\hat{q}, \varepsilon)$ by bisection (Algorithm 1 of the notes) and check it against `kl_inv_upper`.
2. For $m \in \{100, 1000\}$, $\delta = 0.05$, $\varepsilon = \ln(1/\delta)/m$ and $\hat{q} \in \{0, 0.01, 0.05, 0.1, 0.3, 0.5\}$, compare three upper confidence bounds on the risk of a *fixed* classifier: Hoeffding $\hat{q} + \sqrt{\varepsilon/2}$, the kl inversion, and the refined Pinsker relaxation $\hat{q} + \sqrt{2\hat{q}\varepsilon} + 2\varepsilon$. When does the kl make a difference?
3. On `australian`, split the data in two halves. Learn a decision tree of depth 3 on the first half and compute the Hoeffding and kl test-set bounds of its risk on the second half.
4. Now learn trees of depths $1, \dots, 10$ on the first half, keep the one with the smallest error on the second half, and compute its kl bound as if it had been fixed in advance. Why is this “certificate” not valid? Correct it.

Solution

1. The bisection keeps an interval $[a, b]$ with $\text{kl}(\hat{q}||a) \leq \varepsilon < \text{kl}(\hat{q}||b)$, starting from $[\hat{q}, 1]$; since $p \mapsto \text{kl}(\hat{q}||p)$ is increasing on $[\hat{q}, 1]$, halving the interval 30 times gives a precision of 10^{-9} :

```
def kl_inv_bisection(q, eps, tol=1e-9):
    a, b = q, 1.0
    while b - a > tol:
        c = (a + b) / 2
        if kl_bin(q, c) > eps: b = c
        else: a = c
    return b
```

It agrees with `kl_inv_upper` to 10^{-6} on all the values of question 2.

2. Results ($\delta = 0.05$):

m	$\hat{q}$	Hoeffding	kl	refined Pinsker
100	0.00	0.122	0.030	0.060
100	0.01	0.132	0.056	0.094
100	0.05	0.172	0.122	0.165
100	0.10	0.222	0.189	0.237
100	0.30	0.422	0.418	0.494
100	0.50	0.622	0.621	0.733
1000	0.00	0.039	0.003	0.006
1000	0.01	0.049	0.020	0.024
1000	0.05	0.089	0.069	0.073
1000	0.10	0.139	0.125	0.130
1000	0.30	0.339	0.336	0.348
1000	0.50	0.539	0.539	0.561

The Hoeffding width $\sqrt{\ln(1/\delta)/(2m)}$ does not depend on $\hat{q}$ (0.122 for $m = 100$, 0.039 for $m = 1000$). The kl interval is much narrower when $\hat{q}$ is small: for $\hat{q} = 0$ it is $1 - e^{-\varepsilon} \approx \varepsilon$, i.e. a width in $1/m$ instead of $1/\sqrt{m}$ (0.003 instead of 0.039 for $m = 1000$). When $\hat{q}$ approaches $1/2$ the two bounds coincide, as predicted by Pinsker’s inequality, which is tight around $1/2$. The refined Pinsker bound is always between the kl and a looser bound, and captures the fast rate. Conclusion: the kl matters for good classifiers, which is precisely the case of interest.

3. With $n = 345$ test examples, the depth-3 tree has a test error of 0.142, the Hoeffding bound gives 0.208 and the kl bound 0.192 (first numbers of the table of question 4).

4.

n	error (depth 3)	Hoeffding	kl	selected depth	its error	naive kl	kl with $\delta/10$
345	0.142	0.208	0.192	8	0.139	0.189	0.207

The selected depth has been chosen because its error on the second half is the smallest: that half is no longer independent of the selected classifier, and the test-set bound, which holds for a classifier *fixed in advance*, does not apply (selection bias, Figure 10 of the notes). The correction is the Occam bound with a uniform prior on the 10 candidates, i.e. a union bound: replace δ by $\delta/10$, which increases the certificate from 0.189 to 0.207. The price, $\ln 10 \approx 2.3$ nats, is small; the error would be to pay nothing.

Part 3 – Certificates of a standard random forest (Sections 5 and 7.2)

For each dataset, repeat 5 times: split the data into 70% training / 30% test (stratified) and learn a **BaggedTrees** forest of 100 trees on the training set with bootstrap samples of size $m/2$.

1. Compute the test risk of the uniform majority vote, the test Gibbs risk and disagreement, and the three OOB certificates (first-order, tandem, C-bound) of the uniform posterior, with $\delta = 0.05$. Report also the sizes n_1 (smallest OOB set) and n_2 (smallest intersection of two OOB sets).
2. Check that the certificates are always above the test risk. Which bound is the tightest, on which datasets? Relate your answer to the disagreement of the trees (Proposition 5.7) and to n_1, n_2 .
3. Compare the test risk of **BaggedTrees** with `sklearn.ensemble.RandomForestClassifier` (full bootstrap). How much accuracy do we lose by using bootstrap samples of size $m/2$?

Solution

Mean values over the 5 splits (RF: `RandomForestClassifier`; MV, Gibbs, d : uniform forest on the test set; FO, TND, C: OOB certificates of the uniform vote):

dataset	RF	MV	Gibbs	d	n_1	n_2	FO	TND	C-bound
australian	0.137	0.141	0.234	0.258	280	154	0.649	0.819	0.839
balance	0.114	0.097	0.171	0.205	253	138	0.523	0.664	0.731
bupa	0.260	0.254	0.401	0.425	136	70	1.000	1.000	1.000
german	0.245	0.242	0.346	0.364	408	226	0.864	1.000	0.968
heart	0.153	0.158	0.280	0.323	106	54	0.934	1.000	0.996
iono	0.074	0.077	0.158	0.190	139	72	0.560	0.790	0.792
pima	0.250	0.249	0.331	0.327	310	171	0.848	1.000	0.965
sonar	0.194	0.216	0.351	0.394	81	39	1.000	1.000	1.000
wdbc	0.056	0.061	0.093	0.100	231	124	0.324	0.476	0.528
spambase	0.051	0.056	0.124	0.156	1919	1119	0.306	0.289	0.380
abalone8	0.142	0.141	0.211	0.205	1741	1017	0.502	0.591	0.639
pageblocks	0.023	0.021	0.042	0.048	2283	1339	0.124	0.150	0.190
satimage	0.059	0.062	0.100	0.111	2695	1577	0.248	0.271	0.329
segmentation	0.024	0.027	0.070	0.093	958	551	0.210	0.221	0.310
yeast3	0.050	0.046	0.086	0.101	612	346	0.274	0.352	0.424

1–2. All certificates are above the test risk, as they must be (a violation would happen with probability at most 5% per split). Their quality depends above all on the number of OOB examples. On the small datasets (**bupa**, **sonar**, **heart**: $n_2 < 75$) all certificates are vacuous or nearly so: with 54 examples, $\ln(2\sqrt{n}/\delta)/n \approx 0.09$ even with $KL = 0$, and

the factor 2 or 4 does the rest. On the larger datasets (**spambase**, **pageblocks**, **satimage**, **segmentation**) they become informative, e.g. 0.124 for **pageblocks** whose vote errs on 2.1% of the test points.

Which bound is the tightest? Proposition 5.7 says that, as *oracle* quantities, the C-bound and the tandem bound beat the first-order bound when $d \geq R(G)$. This condition holds on almost all datasets (compare the columns Gibbs and d), but the certificates tell a different story: the first-order certificate is the smallest on every dataset except **spambase** (on **bupa** and **sonar** all three are vacuous). The reason is statistical: the tandem bound is estimated on $n_2 \approx 0.55 n_1$ examples and multiplied by 4, and the C-bound needs two kl inversions. Only when the sample is large and the trees diverse enough (**spambase**: $d = 0.156 > R = 0.124$, $n_2 = 1119$) does the tandem certificate win. The lesson: the best oracle bound is not necessarily the best certificate.

3. The two forests have very close test risks (e.g. 0.051 vs 0.056 on **spambase**, 0.137 vs 0.141 on **australian**); on some datasets **BaggedTrees** is even slightly better. Halving the bootstrap samples costs little accuracy, while it raises the OOB proportions from $e^{-1} \approx 37\%$ to $e^{-1/2} \approx 61\%$ (and from 14% to 37% for pairs), which makes the certificates much tighter.

Part 4 – Learning the weights (Sections 6 and 7.2)

1. On each dataset, compute the posteriors minimizing the first-order bound (**optimize_fo**) and the tandem bound (**optimize_tnd**). Report the test risk of the corresponding votes, their certificates and the *effective number of trees* $1/\sum_t \rho_t^2$.
2. Produce a table with one row per dataset and, for each posterior (uniform, FO, TND), the mean and standard deviation over the 5 splits of the test risk and of the certificate. For the uniform posterior, report the best of the three certificates: what correction does this choice require?
3. Which posterior gives the best vote? the best certificate? Are the conclusions of notebook 4 confirmed?

Solution

Means (standard deviations) over the 5 splits; “eff.” is the effective number of trees. For the uniform vote, the certificate is the minimum of the FO, TND and C-bound certificates computed with $\delta/3$ each (union bound, Proposition 6.3), since the best bound is chosen after seeing them.

dataset	uniform		FO-optimized			TND-optimized		
	test risk	best cert.	test risk	FO cert.	eff.	test risk	TND cert.	eff.
australian	0.141 (0.011)	0.666 (0.006)	0.170 (0.029)	0.509 (0.019)	3.2	0.138 (0.013)	0.772 (0.013)	58.3
balance	0.097 (0.016)	0.540 (0.014)	0.137 (0.025)	0.442 (0.023)	5.6	0.101 (0.020)	0.633 (0.029)	71.8
bupa	0.254 (0.038)	1.000 (0.000)	0.367 (0.066)	0.985 (0.019)	5.0	0.283 (0.049)	1.000 (0.000)	66.8
german	0.242 (0.010)	0.879 (0.011)	0.297 (0.035)	0.761 (0.027)	2.3	0.248 (0.014)	0.998 (0.003)	51.8
heart	0.158 (0.044)	0.964 (0.025)	0.212 (0.040)	0.797 (0.027)	8.0	0.165 (0.040)	1.000 (0.000)	74.4
iono	0.077 (0.026)	0.584 (0.016)	0.079 (0.014)	0.440 (0.020)	7.5	0.075 (0.028)	0.739 (0.022)	76.6
pima	0.249 (0.021)	0.864 (0.023)	0.268 (0.028)	0.765 (0.016)	6.0	0.247 (0.023)	0.999 (0.001)	57.6
sonar	0.216 (0.022)	1.000 (0.000)	0.305 (0.027)	0.897 (0.049)	6.6	0.206 (0.010)	1.000 (0.000)	74.3
wdbc	0.061 (0.014)	0.340 (0.014)	0.070 (0.018)	0.258 (0.024)	10.3	0.060 (0.015)	0.438 (0.018)	72.5
spambase	0.056 (0.005)	0.298 (0.004)	0.096 (0.015)	0.263 (0.007)	1.9	0.054 (0.003)	0.272 (0.005)	46.0
abalone8	0.141 (0.003)	0.507 (0.002)	0.178 (0.030)	0.472 (0.005)	5.3	0.143 (0.002)	0.572 (0.003)	51.3
pageblocks	0.021 (0.002)	0.127 (0.001)	0.034 (0.006)	0.105 (0.002)	3.2	0.022 (0.002)	0.140 (0.004)	46.2
satimage	0.062 (0.002)	0.252 (0.004)	0.094 (0.004)	0.227 (0.005)	2.1	0.061 (0.004)	0.258 (0.006)	47.5
segmentation	0.027 (0.010)	0.216 (0.005)	0.049 (0.013)	0.152 (0.010)	1.6	0.025 (0.009)	0.199 (0.004)	47.8
yeast3	0.046 (0.009)	0.282 (0.008)	0.053 (0.008)	0.220 (0.005)	6.3	0.048 (0.006)	0.325 (0.011)	56.0
mean	0.123	0.568	0.161	0.486		0.125	0.623	

Best vote. The TND-optimized vote is as good as the uniform one (mean 0.125 vs 0.123, differences within one standard deviation on every dataset). The FO-optimized vote is worse on all 15 datasets (only marginally on `iono`; mean 0.161), sometimes by a lot (`bupa`: 0.254 $\rightarrow$ 0.367, `sonar`: 0.216 $\rightarrow$ 0.305). The effective number of trees explains why: the FO posterior keeps between 2 and 10 trees, the TND posterior about 50 to 75. Minimizing a first-order bound selects the individually best trees, which are correlated, and destroys the diversity that makes the vote work.

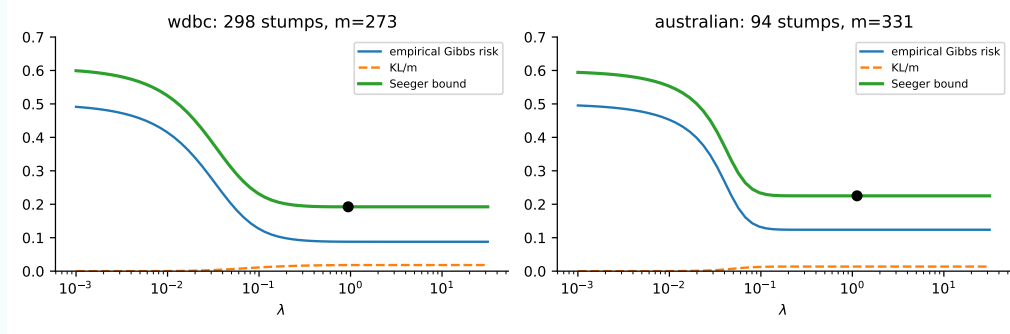
Best certificate. Paradoxically, the smallest certificate is almost always the FO certificate of the FO-optimized posterior (mean 0.486), because it is estimated on more examples and it minimizes precisely this number. But it certifies a worse vote. The TND certificate of the TND-optimized posterior is smaller than (or equal to, when both are vacuous) the TND certificate of the uniform vote on every dataset (e.g. 0.289 $\rightarrow$ 0.272 on `spambase`), and it becomes the best compromise when the data are large enough. This reproduces the conclusions of notebook 4 and of Section 7.2: a certificate that ignores the correlations between voters should not be used as a learning objective for a vote.

Part 5 – The Gibbs posterior and its temperature (Section 4)

We now build the voters ourselves, so that the prior can be fixed before seeing the data used in the bound.

1. On `wdbc` and `australian`, keep 20% of the data to build the voters: all the decision stumps $x \mapsto s \operatorname{sign}(x_j - \theta)$, $s \in \{\pm 1\}$, whose thresholds θ are the quantiles 10, 25, 50, 75, 90% of feature j on this part. Split the remaining data into a sample S (60%) used by the bound and a test sample.
2. With the uniform prior, compute the Gibbs posterior $Q_\lambda \propto e^{-\lambda m \widehat{R}_S(h)}$ for a grid of 60 values of λ between 10^{-3} and $10^{1.5}$, and plot its empirical Gibbs risk, $\text{KL}(Q_\lambda \| P)/m$ and Seeger's bound as functions of λ .
3. Report the best certificate, the corresponding test Gibbs risk and test risk of the vote, and compare with the Occam bound of the stump with the smallest empirical risk. Why is choosing λ on the grid free?

Solution



dataset	n	m	λ	best Gibbs posterior			ERM stump (Occam)			
				$\widehat{R}_S(G)$	KL	Seeger	test Gibbs	test vote	certificate	test
wdbc	298	273	0.943	0.088	4.98	0.192	0.104	0.104	0.196	0.104
australian	94	331	1.124	0.124	4.54	0.225	0.158	0.158	0.225	0.158

Shape of the curves. For small λ the posterior is the prior; since every stump comes with its opposite, the empirical Gibbs risk is exactly 0.5 and the bound is vacuous. When

λ grows the empirical risk drops quickly while KL/m grows slowly, up to $\ln(n)/m$, and the bound reaches a minimum. Beyond this point the KL keeps increasing while the empirical risk no longer decreases.

Numbers. The best certificates are 0.192 (**wdbc**) and 0.225 (**australian**), for test Gibbs risks of 0.104 and 0.158. They are obtained for $\lambda \approx 1$, where the posterior is already concentrated on a few stumps ($KL = 4.98$ and 4.54 nats, close to $\ln n = 5.70$ and 4.54): on these problems a single feature is very informative, and the best Gibbs posterior is nearly the ERM stump, so its certificate is essentially the Occam bound (0.196 and 0.225) and its vote is essentially that stump. On these two problems the Gibbs posterior does as well as Occam; it does better when several voters are about equally good, as with the 300 stumps of **breast cancer** in Section 4 of the notes.

Why is the grid free? λ only indexes posteriors: Seeger's bound holds simultaneously for all Q , hence for all Q_λ , whatever the way λ is chosen (Theorem 6.2). No union bound is needed, contrary to the parameter C of Catoni's bound (Section 6.3).

Part 6 – Strength versus diversity (Section 5)

1. On **spambase** and **german**, learn forests with $\text{max_features} \in \{1, \sqrt{d}, d/2, d\}$ (3 splits). Report the test Gibbs risk, the test disagreement, the test risk of the vote and the OOB tandem and C-bound certificates.
2. Describe how strength and diversity evolve. Which value gives the best vote? Do the certificates follow the risk of the vote?

Solution

dataset	max_features	Gibbs	d	vote	TND cert.	C-bound cert.
spambase	1	0.142	0.192	0.048	0.307	0.413
spambase	$\sqrt{d}$	0.123	0.158	0.055	0.291	0.382
spambase	$d/2$	0.116	0.136	0.060	0.302	0.377
spambase	d	0.114	0.128	0.064	0.309	0.379
german	1	0.362	0.376	0.264	1.000	0.981
german	$\sqrt{d}$	0.346	0.363	0.244	1.000	0.969
german	$d/2$	0.335	0.349	0.234	1.000	0.961
german	d	0.333	0.343	0.242	1.000	0.959

Trying more features per split makes each tree stronger (the Gibbs risk decreases, from 0.142 to 0.114 on **spambase**) but the trees more similar (the disagreement decreases from 0.192 to 0.128). The vote is best with a single feature on **spambase** (0.048) and with $d/2$ features on **german** (0.234): the optimum is a trade-off, as predicted by the C-bound (Figure 19 of the notes). On **spambase** the tandem certificate is minimal at $\sqrt{d}$ (0.291) and the C-bound certificate decreases and then stabilizes: the certificates see the trade-off but not its exact optimum, because their estimation error varies with the configuration. On **german** (700 training examples) the tandem certificate is vacuous and the C-bound certificate hardly informative: with few examples, certificates cannot guide such a fine choice.

Part 7 – Self-bounding model selection (Section 6)

1. On `spambase`, `pima`, `wdbc` and `german`, learn $K = 5$ forests with `max_depth` $\in \{1, 2, 4, 8, \text{None}\}$, minimize the tandem bound of each of them and compute its certificate with δ/K . Select the depth with the smallest certificate.
2. Compare with the depth selected by 5-fold cross-validation of the uniform forest on the training set, and with the best depth in hindsight (smallest test risk). Report the test risks of the selected models.
3. Why must the certificates be computed with δ/K ? When does this selection rule fail, and what should be done then?

Solution

Selected depths on 3 splits, mean test risk of the selected forest, and mean certificate of the model selected by the certificate:

dataset	depth (cert.)	test risk	depth (CV)	test risk	best	certificate
spambase	None, None, None	0.052	None, None, None	0.052	0.052	0.286
pima	1, 1, 1	0.254	4, None, 4	0.238	0.225	1.000
wdbc	8, 8, 8	0.058	4, 4, 8	0.058	0.055	0.489
german	1, 1, 1	0.294	None, None, None	0.247	0.240	1.000

Why δ/K . Keeping the smallest of K certificates is a choice made with the data; the certificate of the selected model is valid only if all K certificates hold simultaneously, which the union bound guarantees (Proposition 6.3). The price is $\ln 5 \approx 1.6$ nats, negligible here.

When it works. On `spambase` and `wdbc`, where the certificates are informative (0.29 and 0.49), the certificate selects deep trees and reaches the same test risk as cross-validation, without any validation fold and with a guarantee on the selected model.

When it fails. On `pima` and `german`, all the certificates equal 1: they are vacuous and carry no information, and the argmin returns the first candidate (depth 1), which is a poor choice (0.254 vs 0.238 for cross-validation on `pima`, 0.294 vs 0.247 on `german`). A selection rule based on certificates is meaningful only when the certificates are non-vacuous: with small samples, one should fall back on cross-validation (and certify the selected model on a held-out sample), or use a first-order or data-dependent-prior certificate, which is less loose on small data.

Part 8 – Imbalanced datasets and the balanced error (Section 6)

The bounds of this course control the 0-1 risk. On imbalanced datasets a trivial classifier predicting the majority class has a small 0-1 risk.

1. On the imbalanced datasets, compare the certificates of the uniform forest with the risk of the trivial classifier. Are the certificates informative?
2. Obtain a certificate on the *balanced* error $\frac{1}{2}(\mathbb{P}(h(x) \neq y | y = +1) + \mathbb{P}(h(x) \neq y | y = -1))$: apply the OOB bounds separately to the training examples of each class (with n_+ and n_- OOB examples) and use a union bound. Implement it and comment.

Solution

Construction. Conditionally on $y = +1$, the examples of the positive class are i.i.d. from $\mathcal{D}_{|y=+1}$, and the OOB construction applies to them: restricting the OOB masks to the positive training examples gives OOB losses and tandem losses whose expectations are the class-conditional error rates. We compute the FO and TND certificates on each class with $\delta/4$ (2 classes $\times$ 2 bounds), keep the smallest on each class, and average them: with

probability $\geq 1 - \delta$ both class-conditional certificates hold, hence their average bounds the balanced error. In code, it suffices to restrict `bt.oob_` to the columns of the class:

```
mask = ytr == cls
saved = bt.oob_; bt.oob_ = saved[:, mask]
L, n1 = bt.oob_losses(Xtr[mask], ytr[mask])
Lt, n2 = bt.oob_tandem(Xtr[mask], ytr[mask])
bt.oob_ = saved
```

	trivial	vote	0-1 cert.	positives: error	cert.	balanced error	balanced cert.
abalone8	0.136	0.141	0.505	0.931	1.000	0.473	0.630
pageblocks	0.102	0.021	0.125	0.110	0.680	0.060	0.380
satimage	0.097	0.062	0.250	0.537	1.000	0.274	0.565
segmentation	0.143	0.027	0.214	0.143	0.837	0.076	0.492
yeast3	0.110	0.046	0.279	0.265	1.000	0.142	0.594

1. The trivial classifier has a 0-1 risk equal to the proportion of positives (0.10 to 0.14). The 0-1 certificates of the forest are larger than this value on every dataset, even on **pageblocks** (0.125 vs 0.102) where the forest is five times better than the trivial classifier on the test set. In other words, the 0-1 certificates cannot even guarantee that the forest is better than predicting the majority class.
2. The balanced error reveals what the 0-1 risk hides: on **abalone8** the vote misses 93% of the positive examples and its balanced error (0.47) is that of a random guess, although its 0-1 risk is 0.141. The balanced certificates are loose, because the positive class contains only a few hundred examples (0.68 on **pageblocks**, vacuous on three datasets), but they are honest: they certify what matters for an imbalanced problem, and they show that informative guarantees on the minority class require many minority examples. A natural improvement is to rebalance the bootstrap samples so that the trees see more positives.

Part 9 – Boosting (Section 7.3)

AdaBoost also outputs a weighted majority vote. To obtain a certificate, the voters must be evaluated on examples independent of them.

1. Split the training set into two halves S_1, S_2 . Run **AdaBoostClassifier** with decision stumps (100 rounds) on S_1 , and consider the learned stumps as the voters, with the uniform prior.
2. Compute on S_2 the FO and TND certificates of the posterior $\rho_t \propto \alpha_t$ given by AdaBoost (with $\delta/2$ each), and of the posteriors minimizing the two bounds. Compare the test risks with those of AdaBoost trained on the whole training set.
3. Why are the certificates of the AdaBoost posterior loose, although AdaBoost is accurate?
Hint: compute the Gibbs risk of the stumps on S_2 and the KL of the AdaBoost posterior.

Solution

dataset	AdaBoost (all data)	AdaBoost (S_1)	AdaBoost posterior on S_2				FO-optimized		TND-optimized	
			Gibbs	KL	FO cert.	TND cert.	test	FO cert.	test	TND cert.
australian	0.148	0.150	0.430	0.30	1.000	1.000	0.147	0.541	0.147	0.908
german	0.245	0.242	0.453	0.39	1.000	1.000	0.301	0.782	0.287	1.000
pima	0.257	0.263	0.443	0.29	1.000	1.000	0.268	0.789	0.247	1.000
spambase	0.064	0.065	0.399	0.29	0.900	0.821	0.204	0.526	0.104	0.561
wdbc	0.043	0.043	0.265	0.03	0.783	0.709	0.103	0.332	0.063	0.465

2. Training on half of the data costs AdaBoost almost nothing ($0.148 \rightarrow 0.150$ on

`australian`). The certificates of its posterior, however, are vacuous on three datasets and loose on the other two (0.71 on `wdbc`). Reweighting the stumps by minimizing the bounds improves the certificates markedly (TND certificate 0.54 on `australian`, 0.33 on `wdbc`), but mostly at the expense of the vote: the TND-optimized vote errs on 20% of `spambase` instead of 6.5%, and on 10% of `wdbc` instead of 4.3%. The optimized posteriors concentrate on the few stumps that are good on their own, and lose the combination built by AdaBoost.

3. The explanation lies in the nature of boosting. Its voters are *weak* on purpose: the Gibbs risk of the stumps on S_2 is 0.40–0.45 on four datasets out of five (0.27 on `wdbc`). AdaBoost makes them accurate *together*, by putting them in a precise combination with large margins, not by making each of them good. The first-order bound is then $2 \times 0.43 > 0.8$, and the tandem bound, which only uses the joint errors of pairs, cannot see a combination that relies on many voters at once. The KL of the AdaBoost posterior is small (0.03 to 0.4 nats), so the complexity is not the problem: the looseness comes from the Gibbs-to-vote relation. This is why boosted votes are better analysed with margin bounds (Section 5.9), or certified with a hold-out sample: on S_2 the test-set bound of the AdaBoost vote itself would be close to its test risk.

Part 10 – Report

Summarize your results in a short report (2–3 pages) containing the tables of Parts 3 and 4, one figure of your choice and a discussion. As in the first practical session, compare methods that are comparable and give standard deviations; remember that a certificate is not an estimate of the risk, and comment on its tightness, not only on its value; finally, report every choice made using the data (grids, splits, selected bounds), since each of them may require a union bound.

Solution

A good report states the three lessons of the session. First, certificates are only informative when the voters are evaluated on enough examples (a few thousand OOB examples for a random forest): on small datasets they are vacuous, and this is not a failure of the implementation. Second, the choice of the bound matters twice: as a certificate, the first-order bound is often the tightest on small and medium datasets and the tandem bound on large ones; as a learning objective, only the second-order bound preserves the diversity and the accuracy of the vote. Third, every data-dependent choice has a price, usually small ($\ln K$), but it must be paid.